

几何作图 DSL - 完整使用文档 (v6)

本文档根据提供的 `DSLInterpreter.ts` 和 `expression.ts` 源代码生成，旨在为该领域特定语言 (DSL) 提供全面的使用指南。

语法规约定

- 指令不区分大小写 (例如 `CREATE POINT` 和 `create point` 等效)。
- 对象名称和插槽变量名区分大小写 (例如 `A` 和 `a` 是不同的点)。
- 所有参数均以 `key=value` 的形式提供。
- 几何指令以 `CREATE` 关键字作为前缀 (例如 `CREATE POINT ...`)。
- 数值参数可以使用直接量 (如 `100`)，也可以引用**插槽(Slot)**中的值。要使用插槽值，请将其名称包含在花括号中，如 `{slot_name}`。支持复杂的数学表达式，例如 `{slot_A * sin(PI()/4)}`。
- 注释以 `#` 开始，直到行尾。

1. 元指令 (Meta Commands)

这些指令用于控制绘图环境、脚本执行流程和输出信息。

1.1 CLEAR

- `CLEAR [color=<color>] [geoColor=<color>] [labelColor=<color>]`
 - 描述:** 清除整个画布，并可以设置后续绘制的默认颜色。
 - 参数:**
 - `color` 或 `c`: (可选) 用于填充画布背景的颜色。默认为 `white`。
 - `geoColor` 或 `g`: (可选) 设置后续 `DRAW` 指令的默认几何图形颜色。默认为 `black`。
 - `labelColor` 或 `l`: (可选) 设置后续 `DRAW` 指令的默认标签颜色。默认为 `black`。
 - 示例:** `CLEAR color=lightblue geoColor=white`

1.2 VIEW

- `VIEW [centerX=<num>] [centerY=<num>] [scale=<num>]`
 - 描述:** 设置画布的视口，实现平移和缩放。
 - 参数:**
 - `centerX`: (可选) 视口中心的逻辑X坐标。默认为 `0`。
 - `centerY`: (可选) 视口中心的逻辑Y坐标。默认为 `0`。
 - `scale`: (可选) 缩放级别。`2`代表放大一倍。默认为 `1`。
 - 示例:** `VIEW centerX=100 centerY=50 scale=2`

1.3 DRAW

- `DRAW obj=<object_name> [options...]`
 - 描述:** 在画布上绘制一个或多个已定义的几何对象。
 - 参数:**
 - `obj=<object_name>`: (必须) 要绘制的对象名称。多个对象可用逗号分隔，例如 `obj=A,B,C`。可以使用 `{slot}` 引用对象名。

- 可选项 (options):
 - `color` 或 `c`: 对象颜色 (例如, `red`, `#FF0000`)。
 - `width`: 线性对象的线宽。
 - `fill`: 闭合图形 (圆、多边形等) 的填充色。
 - `style`: 线条样式 (`solid` 或 `dashed`)。
 - `label` 或 `l`: 为对象添加文本标签。
 - `direction` 或 `d`: 标签相对于对象的位置 (`up`, `down`, `left`, `right`)。默认为`down`。
 - `fontSize` 或 `fs`: 标签的字体大小。
 - `backgroundColor` 或 `bgc`: 标签的背景颜色。
- 示例:
 - `DRAW obj=seg_AB width=3 color=blue style=dashed label=线段AB`
 - `DRAW obj=A,B,C color=red radius=5`

1.4 MEASURE

- `MEASURE type=<type> ... slot=<s>`
 - 描述: 测量几何对象的属性 (如距离、角度、面积) 并将结果存入一个插槽 (Slot) 中。
 - 参数:
 - `type` 或 `t`: (必须) 要测量的属性类型。可选值: `distance` / `length` / `d`, `angle`, `area`。
 - `slot` 或 `s`: (必须) 用于存储测量结果的插槽名称。
 - `p1, p2`: (当`type=distance`时) 用于测量两点间的距离。
 - `obj` 或 `o`: (当`type=length/angle/area`时) 指定要测量的对象。
 - 示例:
 - `MEASURE type=distance p1=A p2=B slot=dist_AB`
 - `MEASURE type=area obj=tri_ABC slot=area_ABC`

1.5 CALCULATE

- `CALCULATE expression=<expr> slot=<s>`
 - 描述: 执行一个数学表达式, 并将结果存入指定的插槽中。
 - 参数:
 - `expression` 或 `e`: (必须) 要计算的数学表达式。
 - `slot` 或 `s`: (可选) 用于存储结果的插槽名称。
 - 示例: `CALCULATE e=sqrt({dist_AB}^2 + {dist_BC}^2) s=hypotenuse`

1.6 GETOBJ

- `GETOBJ name=<obj> property=<prop> slot=<s>`
 - 描述: 从一个对象中获取其某个属性值, 并将该值存入一个插槽中。如果属性本身是另一个几何对象, 则存入的是该几何对象的名称。
 - 参数:
 - `name` / `n` / `object` / `o`: (必须) 源对象的名称。
 - `property` 或 `p`: (必须) 要获取的属性的名称 (例如 `p1`, `center`, `x`, `y`, `radius`)。
 - `slot` 或 `s`: (必须) 用于存储结果的插槽, 如果获取的属性结果是一个数值, 那么该插槽存储的是该数值, 如果结果是一个几何对象, 那么存储的是该几何对象的`name`属性。
 - 示例:
 - `GETOBJ n=tri_ABC p=p1 s=vertex_name`

- `GETOBJ o=A p=x s=A_x`

1.7 PRINT | MESSAGE

- `PRINT | MESSAGE message=<text>`
 - **描述:** `PRINT` 和 `MESSAGE` 指令效果完全相同，用于在控制台或UI消息区打印一条消息，可以包含插槽中的值。
 - **参数:**
 - `message` 或 `m`: (必须) 要显示的文本。**注意:** `message=` 后面的所有内容都将被视为消息文本，直到行尾，因此不需要使用引号。
 - **示例:** `PRINT message=当前距离是 {dist_AB} 个单位。`

1.8 CODE / RUN / WITH

- **描述:** 用于定义、执行和有条件地执行代码块，以组织和重用指令序列。
- `CODE name=<block_name>`: 开始一个命名代码块的定义。后续行直到 `]` 结束都属于该块。
- `RUN code=<block_name>`: 执行一个已定义的代码块。
- `WITH code=<code> with=<slot>`: 当 `with` 指定的插槽值不为零时，执行代码块。
- **示例:**

```
# 定义一个画点的代码块
CODE name=draw_triangle
[
    CREATE POINT name=A x=100 y=0
    CREATE POINT name=B x=-100 y=0
    CREATE POINT name=C x=0 y=100
    CREATE triangle name=ABC p1=A p2=B p3=C draw=true color=red
]
# 直接运行
RUN code=draw_triangle

# 条件性运行
CREATE SLOT name=condition_slot value=1 # 1代表真, 0代表假
CODE name=draw_special_circle
[
    CREATE POINT name=A x=0 y=0
    CREATE CIRCLE name=SpecialCircle center=A radius=50
    DRAW obj=SpecialCircle color=purple
]
# 因为 condition_slot 不为0, 所以下面的代码块会被执行
WITH code=draw_special_circle with={condition_slot}
```

2. 几何指令 (Geometric Commands)

所有几何指令都以 `CREATE` 关键字开头。

重要提示: 所有 `CREATE` 指令都支持一个可选的 `draw=true` 参数。当设置此参数时，新创建的对象会立即被绘制到画布上。您可以在同一行中附加任何 `DRAW` 指令支持的绘图选项（如 `color`, `width`, `label`

等)来控制其样式,如果是一个点,默认会绘制点的Label,与name属性相同,其他几何对象,默认不绘制Label,如需绘制,需要指定参数。

示例: `CREATE POINT name=A x=50 y=50 draw=true color=green radius=5 label=起始点`

2.1 基础对象

• 2.1.1 `CREATE POINT`: name=<name> x=<num> y=<num> [radius=<num>] [real=<bool>]

- 描述: 定义一个点。
- 参数: name(必须), x(必须), y(必须), radius(可选), real(可选,标记是否为“实”点)。
- 示例: `CREATE POINT name=A x=100 y=200 radius=5`

• 2.1.2 `CREATE LINE`: name=<name> p1=<point> p2=<point>

- 描述: 定义一条穿过两个已知点的无限长的直线。
- 示例: `CREATE LINE name=l_AB p1=A p2=B`

• 2.1.3 `CREATE SEGMENT`: name=<name> p1=<point> p2=<point>

- 描述: 定义一条连接两个已知点的线段。
- 示例: `CREATE SEGMENT name=seg_AB p1=A p2=B`

• 2.1.4 `CREATE RAY`: name=<name> vertex=<point> p1=<point>

- 描述: 定义一条从vertex点出发,穿过p1点的射线。
- 参数: name, vertex(或v), p1。
- 示例: `CREATE RAY name=ray_A vertex=A p1=B`

• 2.1.5 `CREATE ANGLE`: name=<name> vertex=<v> p1=<p1> [p2=<p2> | angle=<a>]

- 描述: 定义一个角度。可以通过三点或两点一角度值定义。
- 参数: name, vertex(或v), p1, p2(可选), angle(或a, 可选, 单位:度), showArc(可选,bool)。
- 示例:
 - `CREATE ANGLE name=angle_ABC v=B p1=A p2=C`
 - `CREATE ANGLE name=angle_45 v=O p1=P angle=45`

2.2 派生与构造

• 2.2.1 `CREATE MIDPOINT`: name=<name> p1=<point> p2=<point>

- 描述: 定义两个给定点的中点。
- 示例: `CREATE MIDPOINT name=M p1=A p2=B`

• 2.2.2 `CREATE INTERSECT`: name=<name(s)> obj1=<o1> obj2=<o2>

- 描述: 定义两个几何对象(直线、圆等)的交点。
- 参数: name(或n), obj1(或o1), obj2(或o2)。
- 注意: 如果存在多个交点,请在name参数中使用逗号分隔的名称列表(例如name=P,Q)。
- 示例: `CREATE INTERSECT name=P,Q obj1=my_line obj2=my_circle`

• 2.2.3 `CREATE POINT_ON_LINE`: name=<name> line=<line> point=<p> distance=<d>

- 描述: 在一个线性对象上, 根据与某参考点的距离定义一个新点。
- 参数: `name`, `line`(或`l`), `point`(或`p`), `distance`(或`d`)。
- 示例: `CREATE POINT_ON_LINE name=P3 line=l_AB distance=50 point=A`

- **2.2.4 CREATE PERPENDICULAR_FOOT:** `name=<name> from=<point> on=<line>`

- 描述: 一步到位创建从一个点到一条直线的垂足。
- 参数: `name`(或`n`), `from`(或`f`), `on`(或`o`)。
- 示例: `CREATE PERPENDICULAR_FOOT n=H from=P on=line_AB`

2.3 几何作图 (尺规作图)

- **2.3.1 CREATE PERP_BISECTOR:** `name=<name> p1=<p1> p2=<p2>`

- 描述: 创建连接两点 `p1`, `p2` 的线段的垂直平分线。
- 参数: `name`(必须), `p1`(必须), `p2`(必须)。
- 示例: `CREATE PERP_BISECTOR name=pb_AB p1=A p2=B`

- **2.3.2 CREATE PERPENDICULAR:** `name=<name> point=<p> line=<l>`

- 描述: 创建一条通过指定点 `point` 并垂直于指定直线 `line` 的垂线。
- 参数: `name`(必须), `point`(或`p`, 必须), `line`(或`l`, 必须)。
- 示例: `CREATE PERPENDICULAR name=perp_from_C line=l_AB point=C`

- **2.3.3 CREATE PARALLEL:** `name=<name> point=<p> line=<l>`

- 描述: 创建一条通过指定点 `point` 并平行于指定直线 `line` 的平行线。
- 参数: `name`(必须), `point`(或`p`, 必须), `line`(或`l`, 必须)。
- 示例: `CREATE PARALLEL name=para_from_C line=l_AB point=C`

- **2.3.4 CREATE ANGLE_BISECTOR:** `name=<name> angle=<a>`

- 描述: 创建一个已定义的角度 `angle` 的角平分线。
- 参数: `name`(必须), `angle`(或`a`, 必须)。
- 示例:
 - `CREATE ANGLE name=angle_A ...`
 - `CREATE ANGLE_BISECTOR name=bisector_A angle=angle_A`

2.4 变换

- **2.4.1 CREATE REFLECTED_POINT:** `name=<new> obj=<src> [center=<c> | axis=<a>]`

- 描述: 创建一个点的对称点, 支持中心对称和轴对称。
- 参数: `name`(或`n`), `obj`(或`o`)。 `center`(或`c`)用于中心对称, `axis`(或`a`)用于轴对称。
- 示例:
 - # 中心对称
 - `CREATE REFLECTED_POINT n=A_prime o=A center=O`
 - # 轴对称
 - `CREATE REFLECTED_POINT n=B_prime o=B axis=line_L`

- **2.4.2 CREATE ROTATED_POINT:** `name=<new> obj=<src> center=<c> angle=<a>`

- 描述: 创建一个点绕某中心旋转指定角度后的新点。
- 参数: `name`(或`n`), `obj`(或`o`), `center`(或`c`), `angle`(或`a`, 单位:度)。
- 示例: `CREATE ROTATED_POINT n=B o=A center=O angle=90`

2.5 高级对象

- **2.5.1 CREATE POLYGON:** `name=<name> points=<p1,p2,...>`
 - 描述: 通过一个有序的顶点列表创建一个多边形。
 - 参数: `name`(必须), `points`(或`p`, 必须, 值为逗号分隔的点名列表)。
 - 示例: `CREATE POLYGON name=my_quad points=A,B,C,D`
- **2.5.2 CREATE TRIANGLE:** `name=<name> p1=<p1> p2=<p2> p3=<p3>`
 - 描述: 通过三个顶点创建一个三角形。
 - 参数: `name`(必须), `p1`(必须), `p2`(必须), `p3`(必须)。
 - 示例: `CREATE TRIANGLE name=tri1 p1=A p2=B p3=C`
- **2.5.3 CREATE RECTANGLE:** `name=<name> p1=<p1> width=<w> height=<h>`
 - 描述: 通过左上角顶点 `p1`、宽度和高度创建一个与坐标轴平行的矩形。
 - 参数: `name`(必须), `p1`(必须), `width`(或`w`, 必须), `height`(或`h`, 必须)。
 - 示例: `CREATE RECTANGLE name=rect1 p1=A width=200 height=100`
- **2.5.4 CREATE CIRCLE:** `name=<name(s)> [params...]`
 - 描述: 创建一个或多个圆, 支持多种构造方式。
 - 方法1 (圆心+半径): `center=<c> radius=<r>`
 - `CREATE CIRCLE name=c1 center=O radius=50`
 - 方法2 (圆心+弦): `center=<c> chord=<seg>`
 - `CREATE CIRCLE name=c1 center=O chord=seg_AB`
 - 方法3 (圆心+弦上两点): `center=<c> chordPt1=<p1> chordPt2=<p2>`
 - `CREATE CIRCLE name=c1 center=O chordPt1=A chordPt2=B`
 - 方法4 (弦+圆心角): `chord=<seg> centerAngle=<a>` (可创建两个圆)
 - `CREATE CIRCLE name=c1,c2 chord=seg_AB centerAngle=60`
 - 方法5 (弦上两点+圆心角): `chordPt1=<p1> chordPt2=<p2> centerAngle=<a>`
 - `CREATE CIRCLE name=c1,c2 chordPt1=A chordPt2=B centerAngle=60`
- **2.5.5 CREATE CIRCUMCIRCLE:** `name=<name> p1=<p1> p2=<p2> p3=<p3>`
 - 描述: 创建通过三个给定点的外接圆。
 - 参数: `name`(必须), `p1`, `p2`, `p3` (必须)。
 - 注意: 它同时会自动创建其圆心, 命名为 `<circle_name>_cumcenter`。
 - 示例: `CREATE CIRCUMCIRCLE name=circum p1=A p2=B p3=C`
- **2.5.6 CREATE INCIRCLE:** `name=<name> p1=<p1> p2=<p2> p3=<p3>`
 - 描述: 创建由三个点构成的三角形的内切圆。
 - 参数: `name`(必须), `p1`, `p2`, `p3` (必须)。
 - 注意: 它同时会自动创建其内心, 命名为 `<circle_name>_incenter`。
 - 示例: `CREATE INCIRCLE name=incirc p1=A p2=B p3=C`

- **2.5.7 CREATE TANGENT:** name=<name(s)> circle=<c> point=<p>
 - 描述: 创建从一个点到圆的切线 (或切点)。
 - 参数: name(必须), circle(或c), point(或p)。
 - 注意: 若点在圆上, 则创建一个切点; 若点在圆外, 则创建两个切点, 此时 name 应用逗号分隔两个名称。
 - 示例:
 - CREATE TANGENT name=T circle=c1 point=P_on_circle
 - CREATE TANGENT name=T1,T2 circle=c1 point=P_outside
- **2.5.8 CREATE ELLIPSE:** name=<name> center=<c> radiusX=<rX> radiusY=<rY> [rotation=<rot>]
 - 描述: 创建一个椭圆。
 - 参数: name, center(或c), radiusX(或rX), radiusY(或rY)均为必须; rotation(或rot)为可选。
 - 示例: CREATE ELLIPSE n=E1 c=0 rX=100 rY=50 rot=45
- **2.5.9 CREATE FOCIS:** name=<f1,f2> obj=<ellipse>
 - 描述: 创建一个给定椭圆的两个焦点。
 - 参数: name(必须, 逗号分隔的两个名称), obj(或o, 必须)。
 - 示例: CREATE FOCIS name=F1,F2 obj=my_ellipse
- **2.5.10 CREATE PARABOLA:** name=<name> [params...]
 - 描述: 创建抛物线, 支持两种方式。
 - 方法1 (几何法): vertex=<v> pValue=<p> [rotateAngle=<rot>]
 - CREATE PARABOLA n=P1 v=V p=50 rot=45
 - 方法2 (系数法): a=<a> b= c=<c> (用于 $y=ax^2+bx+c$)
 - CREATE PARABOLA n=P2 a=0.1 b=2 c=5
- **2.5.11 CREATE HYPERBOLA:** name=<name> [params...]
 - 描述: 创建双曲线, 支持两种方式。
 - 方法1 (几何法): center=<c> aValue=<a> bValue= [rotateAngle=<rot>]
 - CREATE HYPERBOLA n=H1 c=0 a=100 b=50
 - 方法2 (焦点法): f1=<f1> f2=<f2> diff=<d>
 - CREATE HYPERBOLA n=H2 f1=F1 f2=F2 diff=80

3. 动态与动画指令

这一系列的指令为您的几何作图带来了生命力。它们允许您创建可以交互、随机化或随时间演变的动态几何图形, 而不仅仅是静态的构造。核心是通过一个名为**插槽(Slot)**的变量系统来实现的。

3.1 CREATE SLOT

- **CREATE SLOT** name=<name> value=<expression>
 - 描述: 创建一个命名的数值变量, 我们称之为“插槽”(Slot)。插槽是实现动态几何的核心, 它可以存储一个数值, 这个数值可以被后续的任何指令通过 {slot_name} 的语法来引用。插槽的值不仅可以是静态的数字, 也可以是依赖于其他插槽的复杂数学表达式的结果。

- 参数:
 - **name**: (必须) 插槽的唯一名称。
 - **value** 或 **v** / **expression** 或 **e**: (必须) 赋给该插槽的初始值或表达式。
- 示例 1: 基本用法

```
# 创建一个名为 radius_val 的插槽，值为 100
CREATE SLOT name=radius_val value=100

# 创建一个圆，其半径引用了上面定义的插槽
CREATE POINT name=O x=200 y=200
CREATE CIRCLE name=C1 center=O radius={radius_val}
DRAW obj=C1
```

- 示例 2: 联动用法

```
# 创建一个基础角度插槽
CREATE SLOT name=base_angle value=45

# 创建一个点，其坐标依赖于插槽中的角度值
# 注意：sin/cos函数使用弧度，所以需要转换
CREATE POINT name=P x={100 * cos({base_angle}*PI()/180)} y={100 *
sin({base_angle}*PI()/180)}
DRAW obj=P color=red
```

3.2 CREATE RANDOMPOINT

- **CREATE RANDOMPOINT** **name**=<name> **obj**=<object> [**start**=<s>] [**end**=<e>]
- 描述: 在一个已存在的几何对象的边界上创建一个随机位置的点。这对于生成随机的几何问题或进行蒙特卡洛模拟非常有用。
- 参数:
 - **name**: (必须) 新创建的随机点的名称。
 - **obj** 或 **o**: (必须) 点将被放置在其上的几何对象的名称。支持 **Line**, **Segment**, **Ray**, **Circle**, **Ellipse** 等。
 - **start** 或 **s**: (可选) 定义随机生成的起始边界。
 - 对于**线性对象** (**Line**, **Segment**, **Ray**)，这是一个从起点开始的比例，范围从 **0** 到 **1**。
 - 对于**圆形/椭圆形对象**，这是一个角度，单位是**度**，范围从 **0** 到 **360**。
 - **end** 或 **e**: (可选) 定义随机生成的结束边界，规则同**start**。
- 示例 1: 在线段上生成随机点

```
CREATE POINT name=A x=100 y=100
CREATE POINT name=B x=400 y=100
CREATE SEGMENT name=segAB p1=A p2=B

# 在线段 segAB 上创建一个随机点 P
CREATE RANDOMPOINT name=P obj=segAB draw=true color=orange radius=5
DRAW obj=segAB
```

◦ 示例 2: 在圆弧上生成随机点

```
CREATE POINT name=O x=250 y=250
CREATE CIRCLE name=C1 center=O radius=150

# 在圆C1的右上角 (0到90度弧线) 上创建一个随机点 Q
CREATE RANDOMPOINT name=Q obj=C1 start=0 end=90 draw=true color=cyan
radius=5
DRAW obj=C1
```

3.3 CREATE ANIMATION

- **CREATE ANIMATION** name=<name> code=<code> slot=<s> [interval=<ms>] [repeat=<bool>]
 - **描述:** 定义并启动一个动画。动画的核心机制是一个定时器，它会以固定的时间间隔 (**interval**) 重复执行一个已定义的**代码块** (**code**)。最关键的是，每执行一次代码块 (即每播放一“帧”)，**CREATE ANIMATION**指令会自动将指定的**插槽** (**slot**) 的值加1。任何依赖于这个插槽的几何对象都会因此而更新位置或形态，从而形成动画。
 - **参数:**
 - **name:** (必须) 为这个动画进程指定一个唯一的名称。
 - **code** 或 **c:** (必须) 在每一帧需要被执行的 **CODE** 代码块的名称。
 - **slot** 或 **s:** (必须) 与动画绑定的插槽名称。该插槽的值会从0开始，每帧自动递增。
 - **interval:** (可选) 每一帧之间的时间间隔，单位是**毫秒**。默认为**1000** (即1秒)。为了流畅动画，建议设为 **16** (约60帧/秒)。
 - **repeat:** (可选) 动画播放到最后是否循环。值为 **true** 或 **false**。默认为**false**。
 - **示例:** 一个从小到大，再从大到小循环的正三角形

```
view centerY=-100
create slot name=slot_ani value=0

CODE name=triangle
[
clear color=black
create slot name=slot_x value={10 * (21 - abs(slot_ani % 40 - 20))}
create point name=A x={0-slot_x} y=0
create point name=B x={slot_x} y=0
create point name=C x=0 y={slot_x * tan(PI()/3)}
create triangle name=ABC p1=A p2=B p3=C draw=true color=red
]

create ANIMATION name=ani1 interval=100 repeat=true code=triangle
slot=slot_ani
```

4. 数学表达式引擎

本DSL拥有一个强大的数学表达式引擎，允许在任何需要数值的参数（如 `x`, `y`, `radius`, `angle` 等）中进行动态计算。

4.1 核心用法：引用插槽(Slot)变量

表达式最基本也是最强大的功能，是引用通过`CREATE SLOT`或`MEASURE`等指令创建的插槽变量。

- **语法:** 将插槽名称放入花括号中，例如 `{my_slot}`。
- **示例:**

```
# 创建一个名为 width 的插槽
CREATE SLOT name=width value=100

# 在表达式中使用该插槽
CREATE POINT name=A x={width / 2} y=50
```

4.2 关键工作流：使用几何对象属性

为了保持语法的简洁和解析的明确性，表达式引擎**不支持**直接访问对象的属性，如 `A.x`。

- **错误用法:**

```
# 这是无效的语法！
CREATE POINT name=B x={A.x + 100} y={A.y}
```

- **正确工作流:** 您必须分两步，先使用 `GETOBJ` 指令将对象的属性提取到一个插槽中，然后再在表达式中引用该插槽。
- **正确示例:**

```
# 假设点A已经存在
CREATE POINT name=A x=50 y=80

# 步骤1：使用 GETOBJ 将点A的x和y坐标分别存入插槽 A_x 和 A_y
GETOBJ obj=A property=x slot=A_x
GETOBJ obj=A property=y slot=A_y

# 步骤2：在表达式中安全地使用这些插槽
CREATE POINT name=B x={{A_x} + 100} y={A_y}
DRAW obj=A,B
```

4.3 支持的运算符

表达式支持标准的算术运算符，并遵循通用优先级规则。

运算符	描述	优先级
<code>^</code>	幂运算	3
<code>*</code>	乘法	2
<code>/</code>	除法	2
<code>%</code>	取模	2
<code>+</code>	加法	1
<code>-</code>	减法	1

4.4 支持的常量

常量	描述	使用示例
<code>PI()</code>	圆周率 π	<code>{2*PI()}</code>
<code>E()</code>	自然常数 e	<code>{E()}</code>

注意：这些常量以无参数函数的形式使用。

4.5 支持的函数

所有三角函数均使用弧度作为参数。

函数名 (别名)	参数个数	描述	示例
三角函数			
<code>sin(rad)</code>	1	正弦	<code>sin({angle_rad})</code>
<code>cos(rad)</code>	1	余弦	<code>cos(PI()/4)</code>
<code>tan(rad)</code>	1	正切	<code>tan(0.5)</code>
<code>cot(rad)</code>	1	余切	<code>cot({angle_rad})</code>
<code>arcsin(val) / asin(val)</code>	1	反正弦	<code>arcsin(0.5)</code>
<code>arccos(val) / acos(val)</code>	1	反余弦	<code>acos(0.5)</code>
<code>arctan(val) / atan(val)</code>	1	反正切	<code>atan(1)</code>
<code>arccot(val) / acot(val)</code>	1	反余切	<code>acot(1)</code>
<code>arctan2(y,x) / atan2(y,x)</code>	2	带象限信息的反正切	<code>atan2({A_y},{A_x})</code>
<code>arccot2(y,x) / acot2(y,x)</code>	2	带象限信息的反余切	<code>acot2({A_y},{A_x})</code>
指数与对数			
<code>exp(x)</code>	1	e 的 x 次幂	<code>exp(2)</code>
<code>log(x)</code>	1	自然对数 (ln)	<code>log({val})</code>

函数名 (别名)	参数个数	描述	示例
<code>sqrt(x)</code>	1	平方根	<code>sqrt(100)</code>
<code>pow(base, exp)</code>	2	幂函数 (等同于 <code>^</code> 运算符)	<code>pow(2, 3)</code>
取整与绝对值			
<code>abs(x)</code>	1	绝对值	<code>abs(-5)</code>
<code>floor(x)</code>	1	向下取整	<code>floor(3.8)</code>
<code>ceil(x)</code>	1	向上取整	<code>ceil(3.2)</code>
其他工具函数			
<code>mod(a,b)</code>	2	模运算 (等同于 <code>%</code> 运算符)	<code>mod(10, 3)</code>
<code>random()</code>	0	生成0到1之间的随机浮点数	<code>100*random()</code>
<code>max(a,b)</code>	2	返回两个数中的最大值	<code>max({val1}, {val2})</code>
<code>min(a,b)</code>	2	返回两个数中的最小值	<code>min(10, 20)</code>